Info Project ¶

-NFL Drafts By 3/c Sidener and 3/c Yoo

Motivation:How do teams know what player is going to be a potential bust or who is going to turn into a star player? We wanted to know whether you can tell whether a player is going to be a bust or star based on where the player was drafted in the draft.

We sought out the data utilizing the website https://www.pro-football-reference.com/draft/ (https://www.pro-football-reference.com/draft/), and focused on players drafted between years 2000-2015. We spent the first 5 days trying to webscrape off of the site itself, but we found a way to download it as an excel file which helped speed up the process. We found the NFL combine data from kaggle: https://www.kaggle.com/kbanta11/nfl-combine/downloads/nfl-combine.zip/1 (https://www.kaggle.com/kbanta11/nfl-combine/downloads/nfl-combine.zip/1) .

We will acquire help regarding statistical analysis through the website:
https://github.com/briandconnelly/BEACONToolkit/blob/master/analysis/scripts/Data%20Analysis%20
(https://github.com/briandconnelly/BEACONToolkit/blob/master/analysis/scripts/Data%20Analysis%2

In [1]:
```
#First we will import the master csv file and pandas and sqlite3(potential qu
#We will also matplotlib and numpy for plotting
import sqlite3
import pandas as pd
import numpy as np
import seaborn as sns

import matplotlib.pyplot as plt
```

In [91]:
```
connection=sqlite3.connect("drafts.db")
cursor=connection.cursor()
```

In [3]:
```
draftpicks=pd.read_csv("Master_draft.csv")
```

We will now check to see if the table has been imported

In [4]:
```
draftpicks.head()
```

Out[4]:

	Rnd	Pick	Tm	Player	Pos	Age	To	AP1	PB	St	...	R.Yds	R.TD	Rec	Re.Yds
0	6	185	ARI	Ryan Lindley	QB	23.0	2015.0	0	0	0	...	7.0	0.0	0.0	0.0
1	7	216	NWE	Owen Pochman	K	24.0	2003.0	0	0	0	...	0.0	0.0	0.0	0.0
2	7	229	DET	Nate Freese	K	24.0	2014.0	0	0	0	...	0.0	0.0	0.0	0.0
3	2	41	HOU	Bennie Joppru	TE	23.0	2007.0	0	0	0	...	0.0	0.0	0.0	0.0
4	2	43	BAL	Sergio Kindle	LB	22.0	2012.0	0	0	0	...	0.0	0.0	0.0	0.0

5 rows × 29 columns

The fundemental question when answering the first question is... what constitutes as a "successful career". Is it having the most passing yards as a QB, is it the QB with the highest completion percentage? Now after doing some research in finding the most unbiased way of determining a "good career", we found that the statistic "Approximate Value" is the most unbiased way to distinguish a player's value. [https://www.sports-reference.com/blog/approximate-value-methodology/ (https://www.sports-reference.com/blog/approximate-value-methodology/)](https://www.sports-reference.com/blog/approximate-value-methodology/)

The approximate value is a relatively non-objective method to compare careers of all positions.

In [5]:
```
# Since we have the CarAv column(standing for career approximate value), we c
# First we will see if we can get the players with the highest carAV
draftpicks_carAV=draftpicks[["Pick","Player","CarAV"]]
draftpicks_carAVsorted=draftpicks_carAV.sort_values(by="CarAV",ascending=Fals
draftpicks_carAVsorted.head()
```

Out[5]:

	Pick	Player	CarAV
3720	199	Tom Brady	175.0
3719	32	Drew Brees	164.0
3718	4	Philip Rivers	140.0
3717	24	Aaron Rodgers	135.0
3716	3	Matt Ryan	129.0

As we can see, these players have had a VERY successful career.

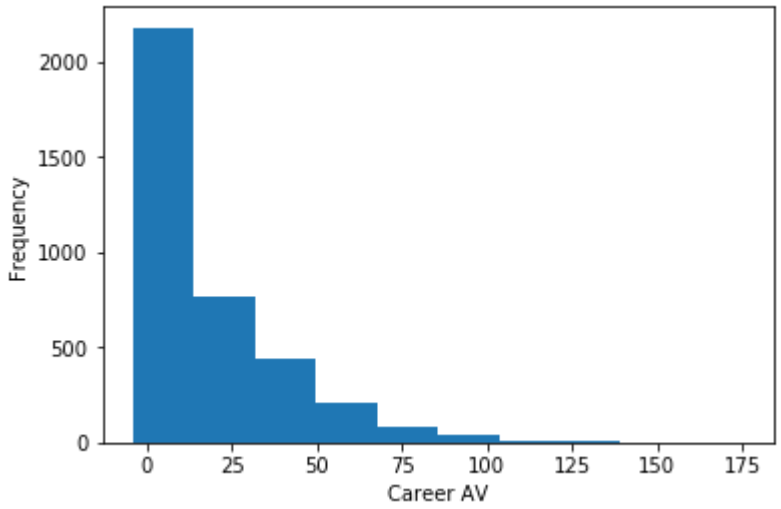
Q1: Does Draft pick correlate with a successful career?

Lets first create a histogram and analyze the overall Career approximate value.

In [6]:
```
%matplotlib inline

draftpicks['CarAV'].plot(kind='hist')
plt.xlabel("Career AV")
```

Out[6]: Text(0.5, 0, 'Career AV')



Out of the 4078 draftees from 2000-2015, around 2300 draftees had less than 12.5 Career Average Value

We can create a scatter plot and create a linear fit line to see the correlation

http://stamfordresearch.com/linear-regression-using-pandas-python/
(http://stamfordresearch.com/linear-regression-using-pandas-python/)

In [7]:

```
%matplotlib inline
scatter_AV_ = draftpicks.plot.scatter(x='Pick', y='CarAV',s=4)
linearplot=np.polyfit(draftpicks.Pick,draftpicks.CarAV,1)
linearplot=linearplot.tolist()
```

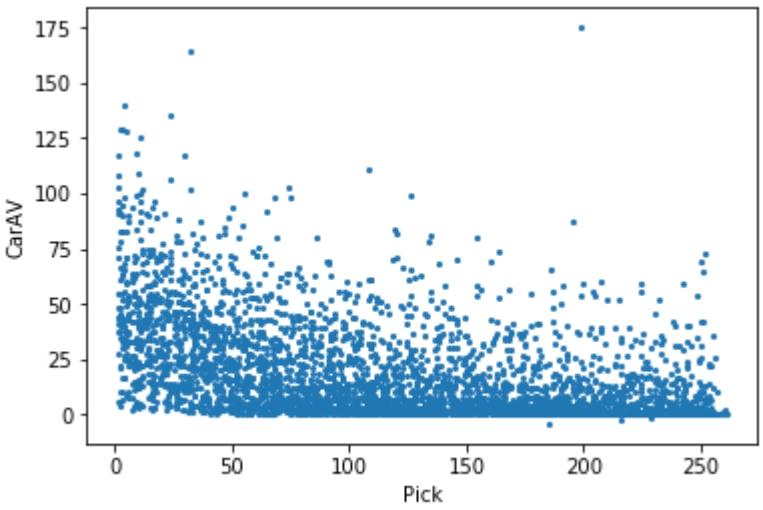
```
---------------------------------------------------------------------------
LinAlgError                               Traceback (most recent call last)
<ipython-input-7-066884d819bc> in <module>
      1 get_ipython().run_line_magic('matplotlib', 'inline')
      2 scatter_AV_ = draftpicks.plot.scatter(x='Pick', y='CarAV',s=4)
----> 3 linearplot=np.polyfit(draftpicks.Pick,draftpicks.CarAV,1)
      4 linearplot=linearplot.tolist()
      5

C:\Anaconda\envs\pycon2019\lib\site-packages\numpy\lib\polynomial.py in pol
yfit(x, y, deg, rcond, full, w, cov)
    626     scale = NX.sqrt((lhs*lhs).sum(axis=0))
    627     lhs /= scale
--> 628     c, resids, rank, s = lstsq(lhs, rhs, rcond)
    629     c = (c.T/scale).T  # broadcast scale coefficients
    630

C:\Anaconda\envs\pycon2019\lib\site-packages\numpy\linalg\linalg.py in lsts
q(a, b, rcond)
   2234         # lapack can't handle n_rhs = 0 - so allocate the array one
larger in that axis
   2235         b = zeros(b.shape[:-2] + (m, n_rhs + 1), dtype=b.dtype)
-> 2236     x, resids, rank, s = gufunc(a, b, rcond, signature=signature, e
xtobj=extobj)
   2237     if m == 0:
   2238         x[...] = 0

C:\Anaconda\envs\pycon2019\lib\site-packages\numpy\linalg\linalg.py in _rai
se_linalgerror_lstsq(err, flag)
    107
    108 def _raise_linalgerror_lstsq(err, flag):
--> 109     raise LinAlgError("SVD did not converge in Linear Least Square
s")
    110
    111 def get_linalg_error_extobj(callback):

LinAlgError: SVD did not converge in Linear Least Squares
```



Now adding a linear approximation to it,

In [8]:

```
scatter_AV_ = draftpicks.plot.scatter(x='Pick', y='CarAV',s=4)
linearplot=np.polyfit(draftpicks.Pick,draftpicks.CarAV,1)
linearplot=linearplot.tolist()


x=np.linspace(0,265,100)
y=linearplot[0]*x+linearplot[1]
plt.plot(x,y,color="red")
```

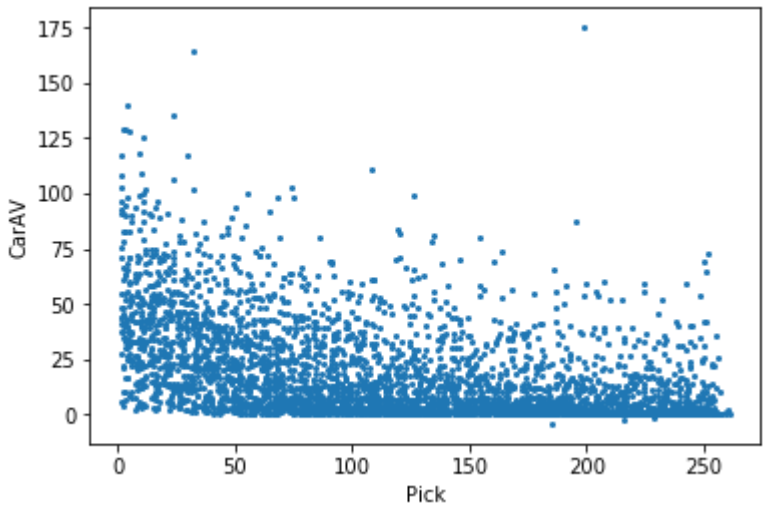
```
---------------------------------------------------------------------------
LinAlgError                               Traceback (most recent call last)
<ipython-input-8-e6d0100003b6> in <module>
      1 scatter_AV_ = draftpicks.plot.scatter(x='Pick', y='CarAV',s=4)
----> 2 linearplot=np.polyfit(draftpicks.Pick,draftpicks.CarAV,1)
      3 linearplot=linearplot.tolist()
      4
      5

C:\Anaconda\envs\pycon2019\lib\site-packages\numpy\lib\polynomial.py in pol
yfit(x, y, deg, rcond, full, w, cov)
    626     scale = NX.sqrt((lhs*lhs).sum(axis=0))
    627     lhs /= scale
--> 628     c, resids, rank, s = lstsq(lhs, rhs, rcond)
    629     c = (c.T/scale).T  # broadcast scale coefficients
    630

C:\Anaconda\envs\pycon2019\lib\site-packages\numpy\linalg\linalg.py in lsts
q(a, b, rcond)
   2234         # lapack can't handle n_rhs = 0 - so allocate the array one
larger in that axis
   2235         b = zeros(b.shape[:-2] + (m, n_rhs + 1), dtype=b.dtype)
-> 2236     x, resids, rank, s = gufunc(a, b, rcond, signature=signature, e
xtobj=extobj)
   2237     if m == 0:
   2238         x[...] = 0

C:\Anaconda\envs\pycon2019\lib\site-packages\numpy\linalg\linalg.py in _rai
se_linalgerror_lstsq(err, flag)
    107
    108 def _raise_linalgerror_lstsq(err, flag):
--> 109     raise LinAlgError("SVD did not converge in Linear Least Square
s")
    110
    111 def get_linalg_error_extobj(callback):

LinAlgError: SVD did not converge in Linear Least Squares
```



In [9]:

```
# we can also look at the correlation coefficient to make sure our findings a
#https://stackoverflow.com/questions/3425439/why-does-corrcoef-return-a-matri
x=draftpicks["Pick"].tolist()
#getting rid of NA values
y=draftpicks.fillna(0.0)["CarAV"].tolist()
corrcoeff=np.corrcoef(x=x,y=y)[1,0]
corrcoeffdouble=corrcoeff**2
print("rvalue:",corrcoeff,",","r^2:",corrcoeffdouble)
```

```
rvalue: -0.5043027395421973 , r^2: 0.2543212531097653
```

There is a weak negative correlation between career AV and pick number. Meaning that as the pick number goes up career AV goes down. However this weak negative correlation is almost negligible as there are plenty of players that have outperformed the norm for their respective pick number.

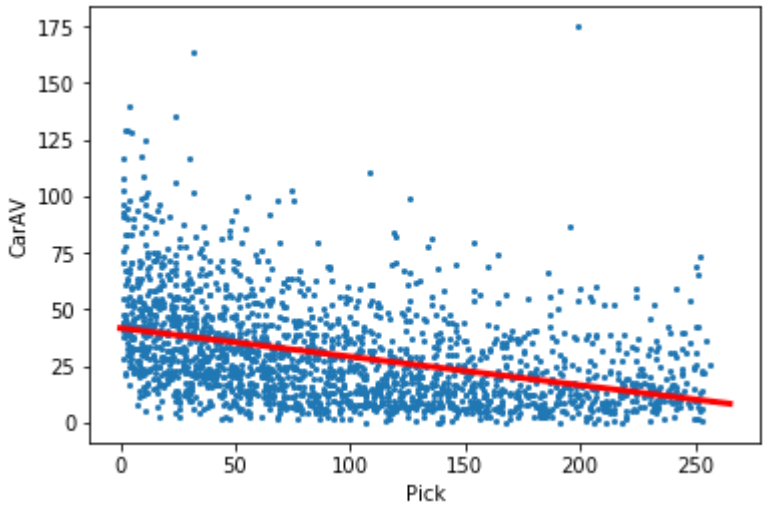
The previous scatterplot included all players who did not even play a single game after getting drafted.

We are going to set it up such that a player must've played at least 48 games minimum (3 season worth) and see if we can create a stronger linear regression plot, and also see an increase in correlation from the r^2.

In [10]:
```
draftpicks_48games=draftpicks[draftpicks["G"]>=48]
scatter_AV_48 = draftpicks_48games.plot.scatter(x='Pick', y='CarAV',s=4)
linearplot=np.polyfit(draftpicks_48games.Pick,draftpicks_48games.CarAV,1)
linearplot=linearplot.tolist()

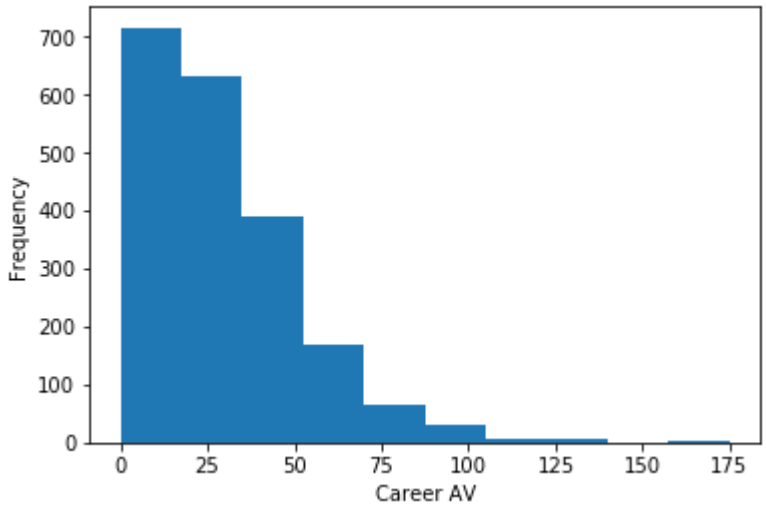

x=np.linspace(0,265,100)
y=linearplot[0]*x+linearplot[1]
plt.plot(x,y,color="red",linewidth=3)
```

Out[10]:
```
[<matplotlib.lines.Line2D at 0x14e125e4c18>]
```



In [11]:
```
draftpicks_48games['CarAV'].plot(kind='hist')
plt.xlabel("Career AV")
```

Out[11]:
```
Text(0.5, 0, 'Career AV')
```



Another look at correlation coefficient and r^2, surprisingly, the correlation between careerAV and draft pick number has actually decreased. Meaning that there draft pick

number doesn't really equate to a successful career.

In [12]:
```
x=draftpicks_48games["Pick"].tolist()

y=draftpicks_48games["CarAV"].tolist()
corrcoeff=np.corrcoef(x=x,y=y)[1,0]
corrcoeffdouble=corrcoeff**2
print("rvalue:",corrcoeff,",","r^2:",corrcoeffdouble)
```

```
rvalue: -0.3976275885130265 , r^2: 0.1581076991466847
```

Q1 Answer: No, according to the statistical correlations, it does not.

Q2: Do teams prioritize certain positions over others in the draft? (First Round only)

A rather simple question, we tried a different angle to the overarching question.

We also had to account for positions that had multiple spots on the roster.

In [13]:

```
%matplotlib inline
draftpicks[["Rnd","Pos"]]
round1_positions=draftpicks[draftpicks["Rnd"]==1]

#round1 draft picks
round1_positions["Pos"].value_counts().plot(kind="Bar",title="First Round")


#Now we have to normalize the data, as different teams have different number
# We also have to combine certain positions together, such as the cornerback
round1=round1_positions.replace(["CB","S"],"DB")
round1_2=round1.replace(["MLB","OLB","ILB"],"LB")
round1_3=round1_2.replace("NT","DT")
round1_4=round1_3.replace(["C","G"],"T")
round1_4["Pos"].value_counts().plot(kind="Bar",title="First Round (UNADJUSTED

#There are no "typical" 53 man nfl roster.
#https://www.quora.com/How-many-players-of-each-type-make-a-typical-53-man-NF
#We are going to use the values here for the the team's 53 man roster and try

#https://stackoverflow.com/questions/14281871/given-a-pandas-series-that-repr

print("Draft Round 1 actual percentages \n" , round1_4["Pos"].value_counts(nc


Position_dict={"QB":[3],"RB":[4],"WR":[7],"TE":[3],"T":[8],"LB":[9],"DB":[7],
Position_dict={"Positions":["QB","RB","WR","TE","T","LB","DB","K","DE","DT"],
position_df=pd.DataFrame(data=Position_dict)

#We tried to normalize the position data by attempting to divide two differen
#we ended up hardcoding the division.

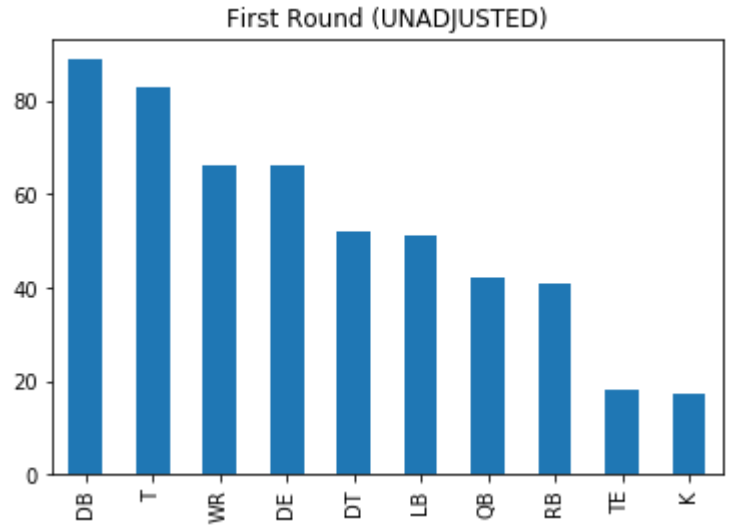
#The purpose was to see which position was picked more than what the expectea


print("\n Expected pick percentage:")
print("QB",position_df["Frequencies"][0]/53)
print("RB",position_df["Frequencies"][1]/53)
print("WR",position_df["Frequencies"][2]/53)
print("TE",position_df["Frequencies"][3]/53)
print("T",position_df["Frequencies"][4]/53)
print("LB",position_df["Frequencies"][5]/53)
print("DB",position_df["Frequencies"][6]/53)
print("K",position_df["Frequencies"][7]/53)
print("DE",position_df["Frequencies"][8]/53)
print("DT",position_df["Frequencies"][9]/53)
```

```
Draft Round 1 actual percentages
 DB    0.174853
T     0.163065
WR    0.129666
DE    0.129666
DT    0.102161
LB    0.100196
QB    0.082515
RB    0.080550
```

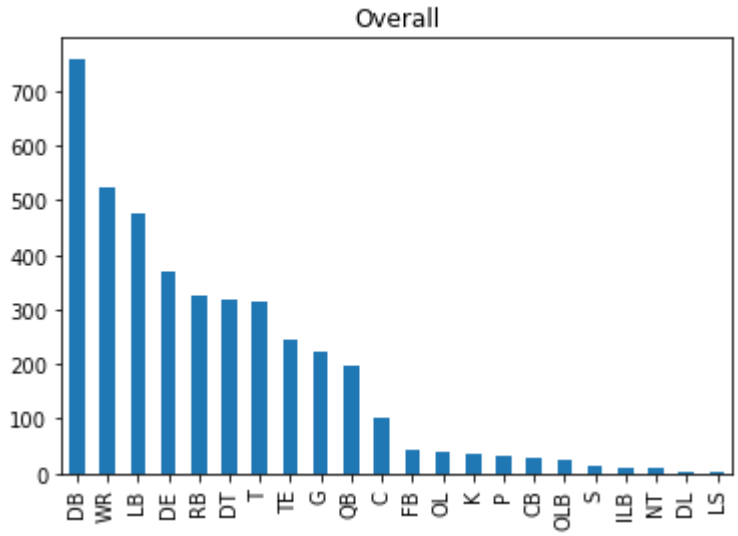
```
TE    0.035363
K     0.001965
Name: Pos, dtype: float64

 Expected pick percentage:
QB 0.05660377358490566
RB 0.07547169811320754
WR 0.1320754716981132
TE 0.05660377358490566
T 0.1509433962264151
LB 0.16981132075471697
DB 0.1320754716981132
K 0.03773584905660377
DE 0.09433962264150944
DT 0.09433962264150944
```



In [14]:
```
draftpicks["Pos"].value_counts().plot(kind="Bar",title="Overall")
```

Out[14]: <matplotlib.axes._subplots.AxesSubplot at 0x14e12b3e128>



Q2 Answer: The largest deviation in expected pick value and actual first round picks came from positions of DB, QB, and DE. This means that teams pick more of these positions early in the draft, which might lead to the conclusion that the position is harder to play, and the talents for the position are lower.

Q3: Are there any distinguishing features from the NFL combine that can play a role in distinguishing good players? (Do athletic abilities actually play a role beyond a certain point?)

In [15]:
```
combineData=pd.read_csv("combine_info.csv")
```

We will try to merge the two dataframes(the draftpick data frame and the combine data frame) to see if how pure athleticism(the combine results) played into the draft picks and

overall career success.

In [16]:
```
#'https://www.datacamp.com/community/tutorials/joining-dataframes-pandas
full_dataframe = pd.merge(combineData, draftpicks, on='Player', how='outer')
full_dataframe
```

Out[16]:

	year	Player	firstname	lastname	position	heightfeet	heightinches	heightinc
0	2015.0	Ameer Abdullah	Ameer	Abdullah	RB	5	9	
1	2015.0	Nelson Agholor	Nelson	Agholor	WR	6	0	
2	2015.0	Jay Ajayi	Jay	Ajayi	RB	6	0	
3	2015.0	Kwon Alexander	Kwon	Alexander	OLB	6	1	
4	2015.0	Mario Alford	Mario	Alford	WR	5	8	
5	2015.0	Javorius Allen	Javorius	Allen	RB	6	0	
6	2015.0	Adrian Amos	Adrian	Amos	FS	6	0	
7	2015.0	Dres Anderson	Dres	Anderson	WR	6	1	

Now we should seek to get rid of all players that have not entered the NFL combine and got drafted

(We can safely assume that no one is shorter than 1 inch.)

In [17]:
```
full_dataframe1= full_dataframe[full_dataframe["G"]>=48]
combine_drafted=full_dataframe1[full_dataframe1["heightinchestotal"]>=1]
```

Since there are many positions that we can compare, lets look at three positions specifically, the running back and the quarter back and the defensive tackle.

In [18]:
```
combine_drafted_RB=combine_drafted[combine_drafted["position"]=="RB"]
combine_drafted_RB
combine_drafted_QB=combine_drafted[combine_drafted["position"]=="QB"]
combine_drafted_QB
combine_drafted_DT=combine_drafted[combine_drafted["position"]=="DT"]
combine_drafted_DT
```

. . .

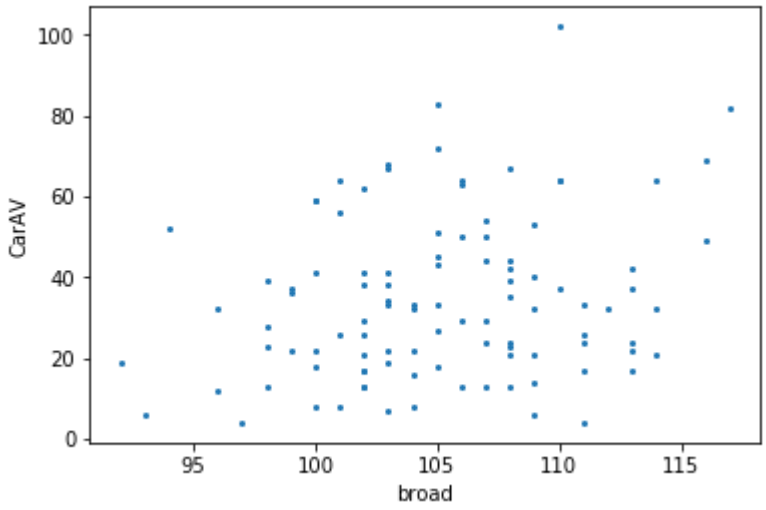
Also, since there are tons of different statistics, we will use the ones that are most prevalent for each position

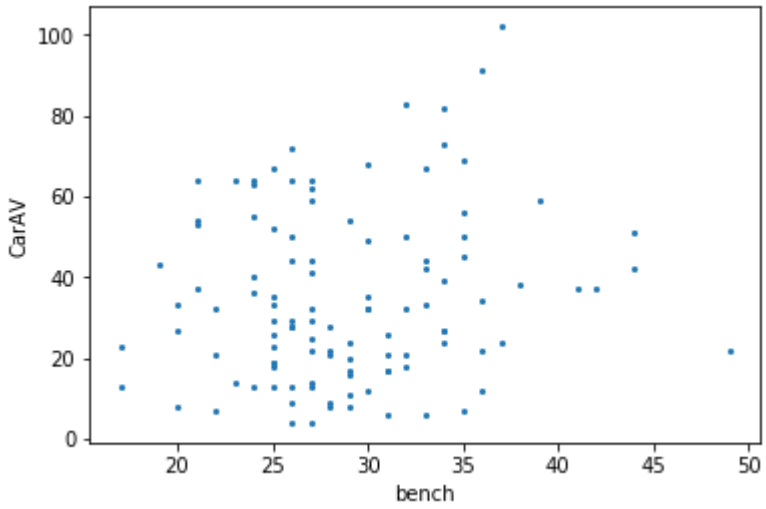
Lets compare their bench, 40yd dash, and broad jump for the Defensive Tackle.

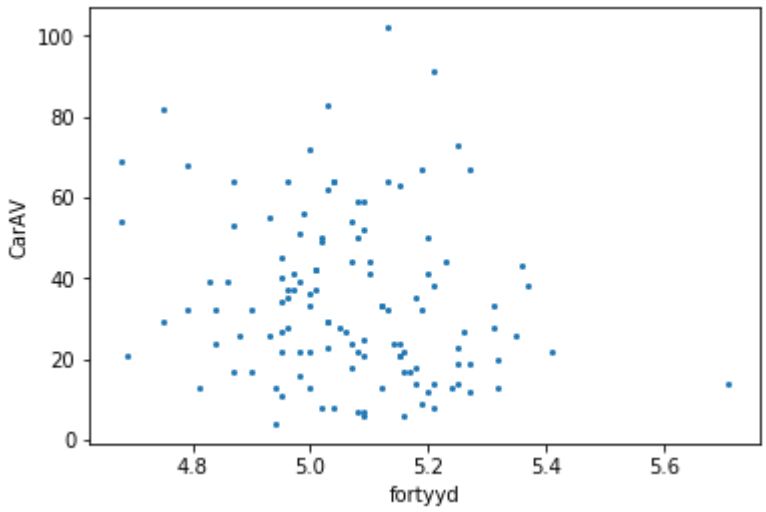
In [19]:

```
%matplotlib inline
combine_drafted_DT_noZeroBroad=combine_drafted_DT[combine_drafted_DT["broad"]
combine_drafted_DT_noZeroBench=combine_drafted_DT[combine_drafted_DT["bench"]
combine_drafted_DT_noZeroforty=combine_drafted_DT[combine_drafted_DT["fortyyd

scatter_DT_broad=combine_drafted_DT_noZeroBroad.plot.scatter(x="broad",y="Car
scatter_DT_bench=combine_drafted_DT_noZeroBench.plot.scatter(x="bench",y="Car
scatter_DT_40=combine_drafted_DT_noZeroforty.plot.scatter(x="fortyyd",y="CarA
```







Lets compare their broad jump, 40yd dash, and bench for the runningback.

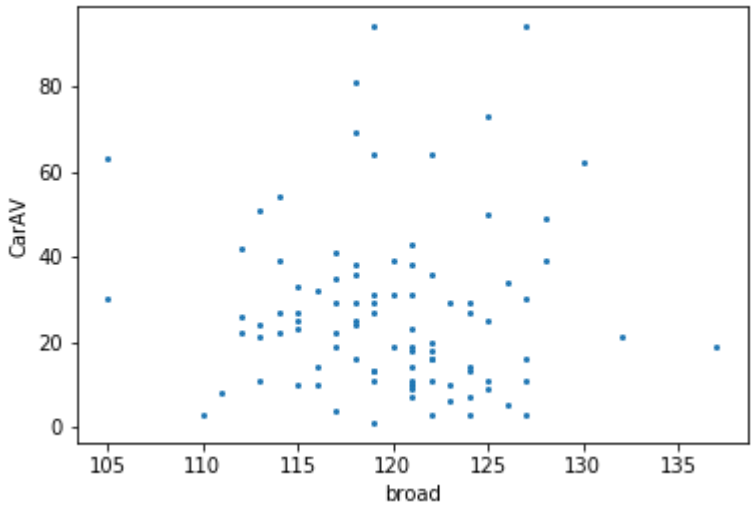
In [109]:

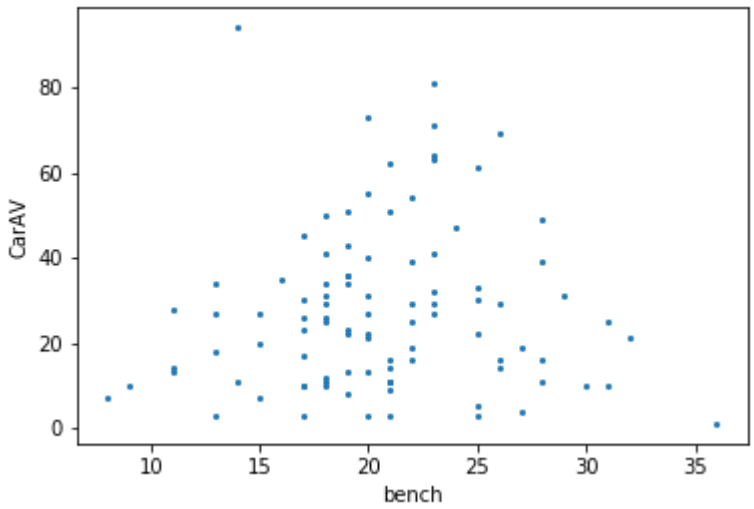
```
combine_drafted_RB_noZerobroad=combine_drafted_RB[combine_drafted_RB["broad"]
combine_drafted_RB_noZeroBench=combine_drafted_RB[combine_drafted_RB["bench"]
combine_drafted_RB_noZeroforty=combine_drafted_RB[combine_drafted_RB["fortyyd

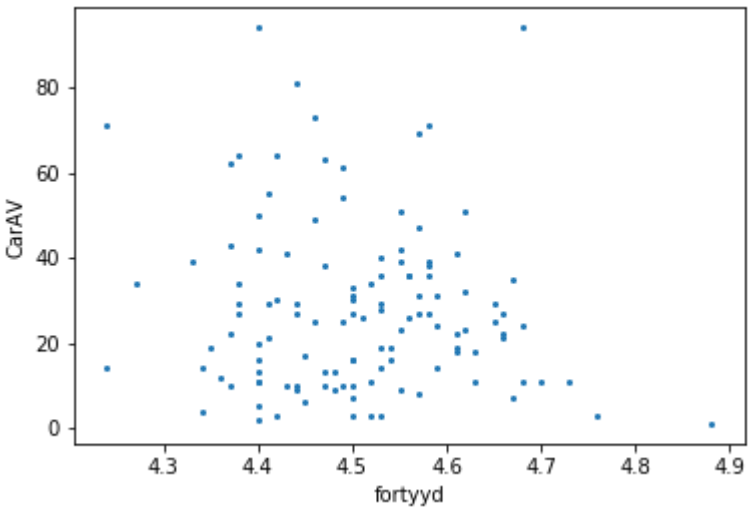

scatter_RB_broad=combine_drafted_RB_noZerobroad.plot.scatter(x="broad",y="Car


scatter_RB_bench=combine_drafted_RB_noZeroBench.plot.scatter(x="bench",y="Car


scatter_RB_40=combine_drafted_RB_noZeroforty.plot.scatter(x="fortyyd",y="CarA
```







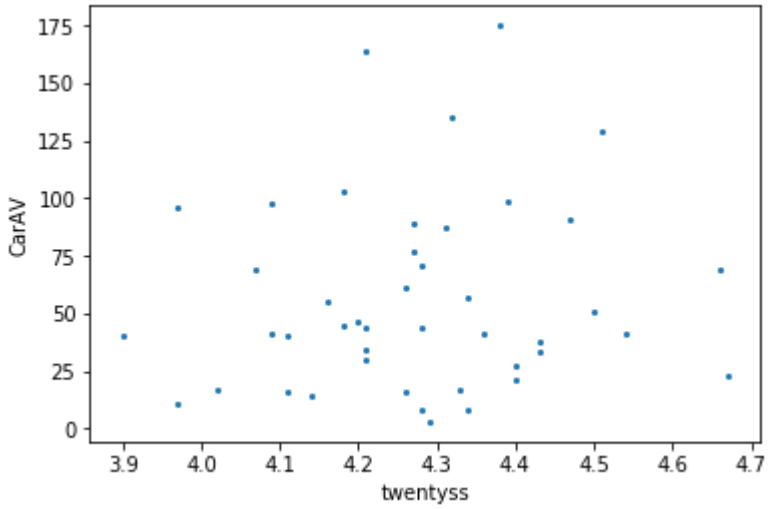
For the QB: 40 yrd dash, 20 yard shuffle, and their broad jump

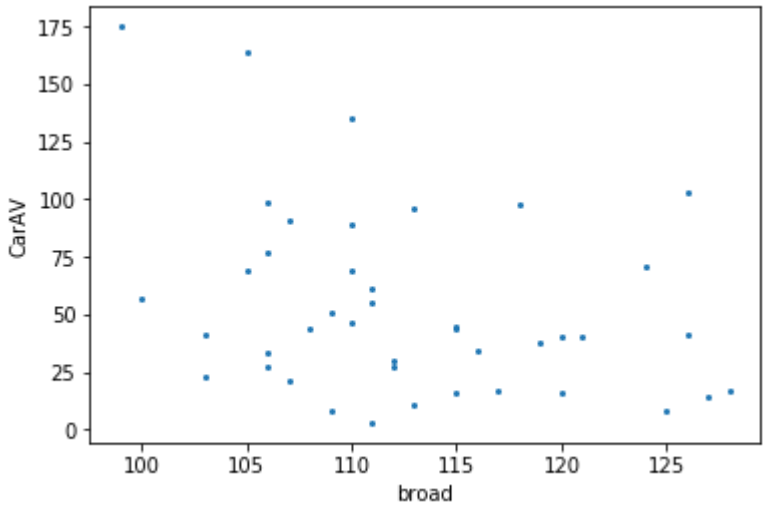
We excluded the bench press because the QBs did not have sufficient bench data

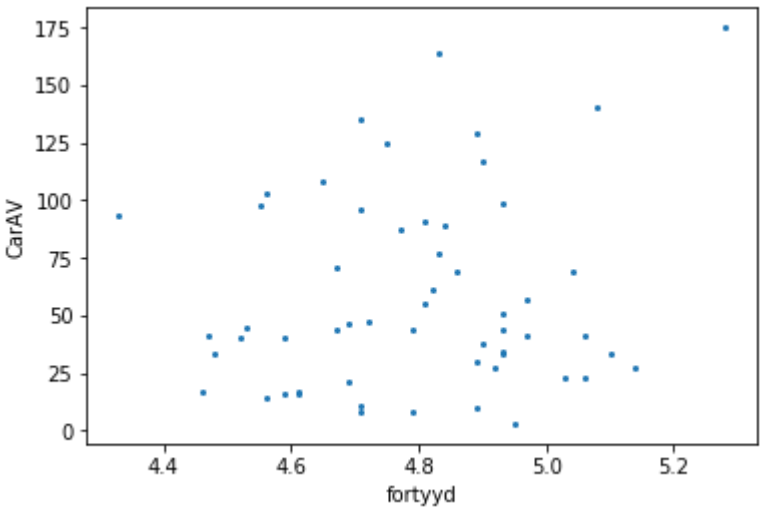
In [110]:

```
combine_drafted_QB_noZerobroad=combine_drafted_QB[combine_drafted_QB["broad"]
combine_drafted_QB_noZero20=combine_drafted_QB[combine_drafted_QB["twentyss"]
combine_drafted_QB_noZeroforty=combine_drafted_QB[combine_drafted_QB["fortyyd

scatter_QB_20=combine_drafted_QB_noZero20.plot.scatter(x="twentyss",y="CarAV"
scatter_QB_broad=combine_drafted_QB_noZerobroad.plot.scatter(x="broad",y="Car
scatter_QB_40=combine_drafted_QB_noZeroforty.plot.scatter(x="fortyyd",y="CarA
```







Now all we have to do is to test and see if there are any correlations between the stats and the career av

In [20]:

```
xList=[combine_drafted_DT_noZeroBroad,combine_drafted_DT_noZeroBench,combine_
yList=["broad","bench","fortyyd"]

for i in range(8):
    xcompare=xList[i]
    r=i%3
    compare=yList[r]
    x=xcompare[compare].tolist()
    y=xcompare["CarAV"].tolist()
    corrcoeff=np.corrcoef(x=x,y=y)[1,0]
    corrcoeffdouble=corrcoeff**2
    if i==0:
        print("DEFENSIVE TACKLE")
    if i==3:
        print("RUNNING BACK")
    if i==6:
        print("QUARTER BACK")
    print("rvalue:" + yList[r],corrcoeff,",","r^2:",corrcoeffdouble)

x=combine_drafted_QB_noZero20["twentyyd"].tolist()
y=combine_drafted_QB_noZero20["CarAV"].tolist()
corrcoeff=np.corrcoef(x=x,y=y)[1,0]
corrcoeffdouble=corrcoeff**2
print("rvalue:"+"20yd shuttle",corrcoeff,",","r^2:",corrcoeffdouble)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-20-59d78e465eab> in <module>
      1
----> 2 xList=[combine_drafted_DT_noZeroBroad,combine_drafted_DT_noZeroBenc
h,combine_drafted_DT_noZeroforty,combine_drafted_RB_noZerobroad,combine_dra
fted_RB_noZeroBench,combine_drafted_RB_noZeroforty,combine_drafted_QB_noZer
obroad,combine_drafted_QB_noZeroforty]
      3 yList=["broad","bench","fortyyd"]
      4
      5 for i in range(8):

NameError: name 'combine_drafted_RB_noZerobroad' is not defined
```

Conclusion for question #3: There really isn't one single statistic from the nfl combine that can determine if a player is going to have a successful career or not... It is a combination of athletic and mental ability.

Q4 What are the differences in Career Approximate Value for different draft rounds?

In [112]:

```
#Hard coded as there are always going to be 7 rounds.
First_Round_Picks=draftpicks[draftpicks["Rnd"]==1]
First_Round_PicksCarAV=First_Round_Picks.sort_values(by="CarAV",ascending=Fal
First_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_1=First_Round_Picks["CarAV"].mean()

Second_Round_Picks=draftpicks[draftpicks["Rnd"]==2]
Second_Round_PicksCarAV=Second_Round_Picks.sort_values(by="CarAV",ascending=F
Second_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_2=Second_Round_Picks["CarAV"].mean()

Third_Round_Picks=draftpicks[draftpicks["Rnd"]==3]
Third_Round_PicksCarAV=Third_Round_Picks.sort_values(by="CarAV",ascending=Fal
Third_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_3=Third_Round_Picks["CarAV"].mean()

Fourth_Round_Picks=draftpicks[draftpicks["Rnd"]==4]
Fourth_Round_PicksCarAV=Fourth_Round_Picks.sort_values(by="CarAV",ascending=F
Fourth_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_4=Fourth_Round_Picks["CarAV"].mean()

Fifth_Round_Picks=draftpicks[draftpicks["Rnd"]==5]
Fifth_Round_PicksCarAV=Fifth_Round_Picks.sort_values(by="CarAV",ascending=Fal
Fifth_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_5=Fifth_Round_Picks["CarAV"].mean()

Sixth_Round_Picks=draftpicks[draftpicks["Rnd"]==6]
Sixth_Round_PicksCarAV=Sixth_Round_Picks.sort_values(by="CarAV",ascending=Fal
Sixth_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_6=Sixth_Round_Picks["CarAV"].mean()

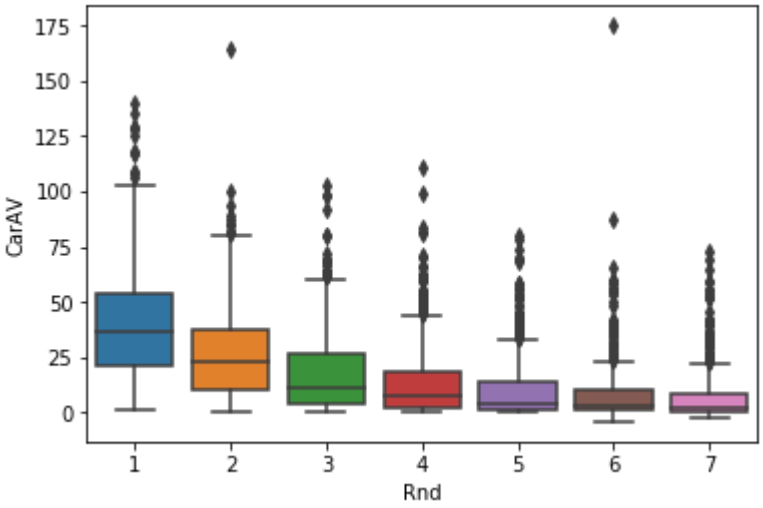
Seventh_Round_Picks=draftpicks[draftpicks["Rnd"]==7]
Seventh_Round_PicksCarAV=Seventh_Round_Picks.sort_values(by="CarAV",ascending
Seventh_Round_PicksCarAV[["Player","CarAV"]]
avg_CarAvg_7=Seventh_Round_Picks["CarAV"].mean()

carAV_Rounds=[avg_CarAvg_1,avg_CarAvg_2,avg_CarAvg_3,avg_CarAvg_4,avg_CarAvg_
Rounds=[1,2,3,4,5,6,7]

plt.bar(Rounds,carAV_Rounds)
plt.ylabel('CarAV', fontsize=20)
plt.xlabel('Pick Rounds', fontsize=20)


%matplotlib inline
#https://www.youtube.com/watch?v=KoTHJPzpw0c
#tips=sns.load_dataset('tips')

boxplot=sns.boxplot(x="Rnd",y="CarAV",data=draftpicks)
```



Q5 Top 5 picks in each draft according to their Career AV

In [21]:

```
draftpicks_Rnd1=draftpicks[draftpicks["Rnd"]==1]
draftpicks_Rnd2=draftpicks[draftpicks["Rnd"]==2]
draftpicks_Rnd3=draftpicks[draftpicks["Rnd"]==3]
draftpicks_Rnd4=draftpicks[draftpicks["Rnd"]==4]
draftpicks_Rnd5=draftpicks[draftpicks["Rnd"]==5]
draftpicks_Rnd6=draftpicks[draftpicks["Rnd"]==6]
draftpicks_Rnd7=draftpicks[draftpicks["Rnd"]==7]


draftpicks_Rnd=[draftpicks_Rnd1,draftpicks_Rnd2,draftpicks_Rnd3,draftpicks_Rn
for i in range (7):
    draftpicks_bestByRound=draftpicks_Rnd[i].sort_values(by="CarAV",ascending
    top5=draftpicks_bestByRound[0:5]
    if i==0:
        print ("\n First Round ")
    if i==1:
        print (" \n Second Round")
    if i==2:
        print ("\n Third Round")
    if i==3:
        print ("\n Fourth Round ")
    if i==4:
        print ("\n Fifth Round ")
    if i==5:
        print ("\n Sixth Round")
    if i==6:
        print ("\n Seventh Round")
    print(top5[["Pick","Player","CarAV"]])
```

```
 First Round
      Pick                   Player  CarAV
3718     4            Philip Rivers  140.0
3717    24            Aaron Rodgers  135.0
3716     3                Matt Ryan  129.0
3715     2           Julius Peppers  129.0
3714     5  LaDainian Tomlinson HOF  128.0

 Second Round
      Pick             Player  CarAV
3719    32        Drew Brees  164.0
3701    55  Andrew Whitworth  100.0
3690    50   Calais Campbell   94.0
3677    48        Matt Light   89.0
3673    36      Chad Johnson   87.0

 Third Round
      Pick           Player  CarAV
3705    74      Steve Smith  103.0
3697    75  Russell Wilson   98.0
3696    68    Lance Briggs   98.0
3686    65      Frank Gore   92.0
3655    86   Marshal Yanda   80.0
```

Fourth Round

	Pick	Player	CarAV
3709	108	Jahri Evans	111.0
3699	126	Jared Allen	99.0
3668	119	Brandon Marshall	84.0
3662	120	Geno Atkins	82.0
3659	135	Josh Sitton	81.0

Fifth Round

	Pick	Player	CarAV
3656	154	Richard Sherman	80.0
3650	134	Kyle Williams	78.0
3638	164	Dan Koppen	74.0
3611	146	Trent Cole	70.0
3607	160	David Diehl	69.0

Sixth Round

	Pick	Player	CarAV
3720	199	Tom Brady	175.0
3674	195	Antonio Brown	87.0
3586	186	Adalius Thomas	66.0
3536	207	Antoine Bethea	60.0
3528	200	Chris Myers	59.0

Seventh Round

	Pick	Player	CarAV
3635	252	Marques Colston	73.0
3608	250	Ryan Fitzpatrick	69.0
3581	251	Scott Wells	65.0
3530	242	Brett Keisel	59.0
3529	224	Mark Tauscher	59.0

Conclusion: We couldn't find any sort of correlation between draft picks and their overall career success in the nfl. However, through this data analysis, we were able to realize the complexities and intricacies that play into the world of sport statistics. We also realized that given more time, we could look further into trying to find a specific undervalued statistic that can correlate with a successful career in the nfl, and actively trying to look for players who have been undervalued. Our understanding of python and data analysis has grown immensely through this project.

Type *Markdown* and LaTeX: α^2